

Question No:9

DS	Session	Tree 1	Question Information	Level 1	Challenge 69
Question description A forest is an undirected graph without cycles (not necessarily connected). Mohana and john are friends in kerala, both of them have a forest with nodes numbered from 1 to n, and they would like to add edges to their forests such that: • After adding edges, both of their graphs are still forests. • They add the same edges. That is, if an edge (u,v) is added to Mohana's forest, then an edge (u,v) is added to john's forest, and vice versa. Mohana and johns want to know the maximum number of edges they can add, and which edges to add. Constraints: $1 \leq n \leq 105,$ $0 \leq m1$ $m2 < n$ $1 \leq u, v \leq n, u \neq v$ Input The first line contains three integers $n, m1$ and $m2$ — the number of nodes and the number of initial edges in Mohana's forest and john's forest. Each of the next $m1$ lines contains two integers u and v — the edges in Mohana's forest. Each of the next $m2$ lines contains two integers u and v ($1 \leq u, v \leq n, u \neq v$) — the edges in Johns's forest.					

Output

The first line contains only one integer h , the maximum number of edges Mohana and John can add.

Each of the next h lines contains two integers u and v ($1 \leq u, v \leq n$, $u \neq v$) — the edge you add each time.

If there are multiple correct answers, you can print any one of them.

✓ Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 3 2
5 4
2 1
4 3
4 3
1 4
```

EXPECTED OUTPUT

```
1
1 5
```

Test Case 2

INPUT (STDIN)

```
5 3 2
5 3
2 3
2 3
2 3
2 4
```

EXPECTED OUTPUT

```
1
1 2
4 5
```

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #define MAXN 100005
5
6  struct DSU {
7      int parent[MAXN];
8      int size[MAXN];
9  };
10
11 struct Edge {
12     int u;
13     int v;
14 };
15
16 void init(struct DSU *d, int n) {
17
18     for (int i = 1; i <= n; i++) {
19
20         d->parent[i] = i;
21         d->size[i] = 1;
22     }
23 }
24
25 int find(struct DSU *d, int x) {
26
27     while (d->parent[x] != x) {
28
29         d->parent[x] =
```

```
29         d->parent[x] =
30         |         d->parent[d->parent[x]];
31
32         x = d->parent[x];
33     }
34
35     return x;
36 }
37
38 int unite(struct DSU *d, int a, int b) {
39
40     a = find(d, a);
41     b = find(d, b);
42
43     if (a == b)
44         return 0;
45
46     if (d->size[a] < d->size[b]) {
47
48         int temp = a;
49         a = b;
50         b = temp;
51     }
```

```
53     d->parent[b] = a;
54     d->size[a] += d->size[b];
55
56     return 1;
57 }
58
59 int same(struct DSU *d, int a, int b) {
60
61     return find(d, a) == find(d, b);
62 }
63
64 int main() {
65
66     int n, m1, m2;
67
68     scanf("%d %d %d",
69         &n,
70         &m1,
71         &m2);
72
73     struct DSU d1;
74     struct DSU d2;
75
76     init(&d1, n);
```

```

77     init(&d2, n);
78
79     /* Mohana's forest */
80
81     for (int i = 0; i < m1; i++) {
82         int u, v;
83
84         scanf("%d %d", &u, &v);
85
86         unite(&d1, u, v);
87     }
88
89     /* John's forest */
90
91     for (int i = 0; i < m2; i++) {
92         int u, v;
93
94         scanf("%d %d", &u, &v);
95
96         unite(&d2, u, v);
97     }
98
99
100
101     struct Edge *answer =
102         malloc((n - 1) * sizeof(struct Edge));
103

```

```
103
104     int count = 0;
105
106     /*
107      First connect city 1 to vertices which
108      are disconnected from 1 in BOTH forests.
109     */
110
111     for (int v = 2; v <= n; v++) {
112
113         if (!same(&d1, 1, v) &&
114             !same(&d2, 1, v)) {
115
116             answer[count].u = 1;
117             answer[count].v = v;
118
119             count++;
120
121             unite(&d1, 1, v);
122             unite(&d2, 1, v);
123         }
```

```
124     }
125
126     /*
127     Collect representatives of the remaining
128     components.
129     */
130
131     int *a =
132         malloc(n * sizeof(int));
133
134     int *b =
135         malloc(n * sizeof(int));
136
137     int *seen1 =
138         calloc(n + 1, sizeof(int));
139
140     int *seen2 =
141         calloc(n + 1, sizeof(int));
142
143     int na = 0;
144     int nb = 0;
145
146     int root1 = find(&d1, 1);
147     int root2 = find(&d2, 1);
148
149     for (int v = 2; v <= n; v++) {
```

```

149     for (int v = 2; v <= n; v++) {
150
151         int r1 = find(&d1, v);
152         int r2 = find(&d2, v);
153
154         if (r1 != root1 &&
155             !seen1[r1]) {
156
157             seen1[r1] = 1;
158             a[na++] = v;
159         }
160
161         if (r2 != root2 &&
162             !seen2[r2]) {
163
164             seen2[r2] = 1;
165             b[nb++] = v;
166         }
167     }
168
169     /*
170     |   Pair components.
171     */
172
173     int k =
174         (na < nb) ? na : nb;
175
176     for (int i = 0; i < k; i++) {

```

```
178     int u = a[i];
179     int v = b[i];
180
181     if (!same(&d1, u, v) &&
182         !same(&d2, u, v)) {
183
184         answer[count].u = u;
185         answer[count].v = v;
186
187         count++;
188
189         unite(&d1, u, v);
190         unite(&d2, u, v);
191     }
192 }
193
194 /*
195  | Output
196  */
197
198 printf("%d\n", count);
199
200 for (int i = 0; i < count; i++) {
201
202     printf("%d %d\n",
```

```

203         answer[i].u,
204         answer[i].v);
205     }
206
207     free(answer);
208     free(a);
209     free(b);
210     free(seen1);
211     free(seen2);
212
213     return 0;
214 }

```

Output:

Test Case-1:

```

5 3 2
5 4
2 1
4 3
4 3
1 4
1
1 5
PS C:\Users\kecha\One

```

Test Case-2:

	5 3 2	
	5 3	
	2 3	
	2 3	
	2 3	
	2 4	
	2	
	1 2	
	4 5	

Keshav Kumar Rauniyar

CH.SC.U4CSE25236